



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Parallel Programming

Prof. Marco Bertini



UNIVERSITÀ
DEGLI STUDI
FIRENZE



Introduction

Parallelism vs. Concurrency

- A system is said to be **concurrent** if it can support two or more actions *in progress* at the same time.
- A system is said to be **parallel** if it can support two or more actions executing at the same time.
- The key concept and difference between these definitions is the phrase “in progress”.



Parallelism vs. Concurrency

- A **concurrent application** will have two or more threads in progress at some time.
E.g.: two threads that are being swapped in and out by the operating system on a single core processor. These threads will be “in progress”—each in the midst of its execution—at the same time.
- A **parallel application** will have two or more threads executing simultaneously if the computation platform has multiple cores available.
E.g.: two or more threads could each be assigned a separate core and would be running simultaneously.

Parallelism vs. Concurrency

- Parallelism $\subseteq$ Concurrency
- A concurrent program becomes parallel if there are enough cores to execute its multiple threads or processes.
- Concurrent programming: composition of independently executing processes.
- Parallel programming: programming as the simultaneous execution of (possibly related) computations.

Parallelism vs. Concurrency

- **Concurrency** is about dealing with lots of things at once.
- **Parallelism** is about doing lots of things at once.
- These are not the same thing, but they are related.
- **Concurrency** is about structure, **Parallelism** is about execution.
- **Concurrency** provides a way to structure a solution to solve a problem that may (but not necessarily) be **parallelizable**.

Parallelism vs. Concurrency

- During this lecture I have to deal with concurrent things: advancing the slides checking that the projector works, delivering the lecture and perhaps breaking to answer a question that arises from the lecture.
- I'm doing only one of these things at a certain time.
- If I'm helped by one or two assistants that check the projector and answer the questions while I'm lecturing then the process will become concurrent and parallel.
- If the assistants and I perform a different task independently, e.g. changing the slides without caring if I'm talking about something different, then the process is parallel.

Parallelism vs. Concurrency

- During this lecture I have to deal with concurrent things:

advancing
delivering
question

A **concurrent** program has **multiple logical threads of control**. These threads may or may not run in parallel.

- I'm doing

- If I'm here
projector
the projector

A **parallel** program potentially runs more quickly than a sequential program by executing different parts of the computation simultaneously (in parallel).

- If the assignment
independent
talking about
parallel.

It may or may not have more than one logical thread of control.

I'm

Sequential algorithm

- Most of today's algorithms are sequential: they specify a sequence of steps in which each step consists of a single operation.
- It has worked so far because of the free performance improvement provided by CPUs and their increase in clock and their pipelining...
- ... but this free ride is over. Chip manufacturers are not working on improving clock speed.



Parallel algorithm

- A parallel algorithm is designed to execute multiple operations at the same step
- The parallelism in an algorithm can yield improved performance on many different kinds of computers.
 - For example, on a parallel computer, the operations in a parallel algorithm can be performed simultaneously by different processors or cores.
 - Even on a single-processor computer the parallelism in an algorithm can be exploited by using multiple functional units, pipelined functional units, or pipelined memory systems.

Parallel algorithm

- A parallel algorithm is designed to execute multiple operations at the same step
- The parallelism in an algorithm can yield improved performance on many different kinds of computers.
 - For example, on a parallel computer, the operations in a parallel algorithm can be performed simultaneously by different processors or cores.

In parallel programming we identify and expose parallelism in algorithms, expressing this in our software, and understanding the costs, benefits, and limitations of the chosen implementation.

Concurrent algorithm

- A concurrent algorithm is an algorithm structured so that it is possible to execute its steps independently.
- A concurrent algorithm can be executed serially.
- There is need of some communication to coordinate the independent executions.



Concurrent algorithm

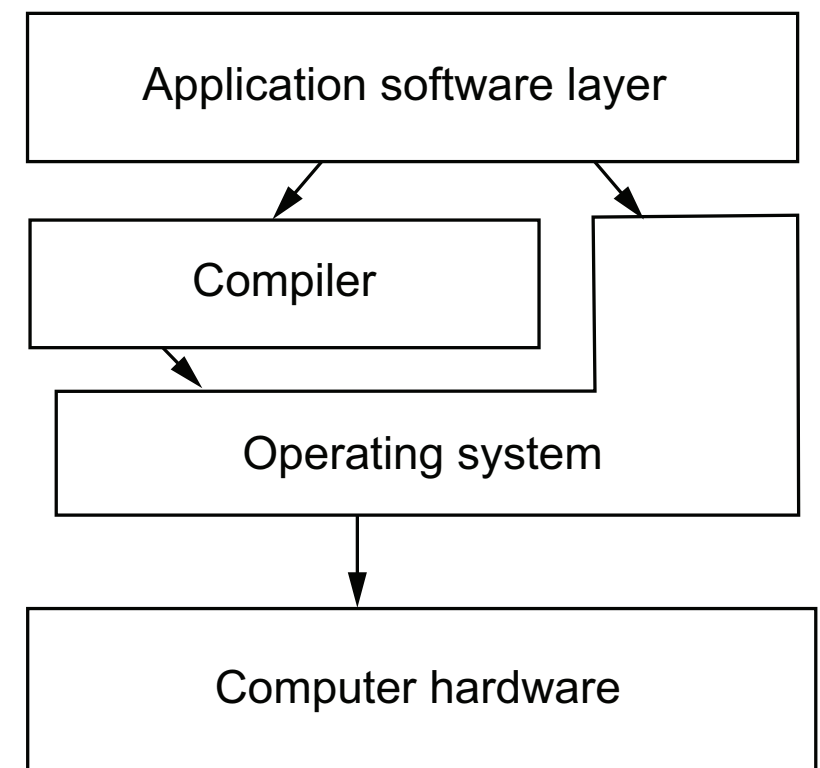
- A concurrent algorithm is an algorithm structured so that it is possible to execute its steps independently.
- A concurrent algorithm can be executed serially.

Since multicore CPUs are now available everywhere we will talk about parallel execution of concurrent code.

We will use the term “parallelization” when dealing with translation of serial code to concurrent (but this should be “concurrentization”)

Parallel Programming

- As a developer, you are responsible for the application software layer, which includes your source code.
- In the source code, you make choices about the programming language and parallel software interfaces you use to leverage the underlying hardware.
- Additionally, you decide how to break up your work into parallel units.
- A compiler is designed to translate your source code into a form the hardware can execute. With these instructions at hand, an OS manages executing these on the computer hardware.



Motivations

- It is not always obvious that a parallel algorithm has benefits, unless we want to do things ...
 - faster: doing the same amount of work in less time
 - bigger: doing more work in the same amount of time
- Both are good outcomes of program parallelization: they improve the results of the program.

Motivations

- Consider a 16-core CPU with hyperthreading and a 256 bit-wide vector unit. How many parallel operations can it perform ?
 - $16 \text{ (cores)} \times 2 \text{ (hyperthreads)} \times 256 \text{ (bit-wide vector unit)} / 64 \text{ (bit-wide double)} = 128\text{-way parallelism}$
- Using only 1 serial path out of 128 parallel paths means that we are using $1/128 = 0.008$ or 0.8% of the available computational capabilities.

Motivations

- Consider a 16-core CPU with hyperthreading and a 256 bit-wide vector unit. How many parallel operations can it perform ?
 - $16 \text{ (cores)} \times 2 \text{ (hyperthreads)} \times 256 \text{ (bit-wide vector unit)} / 64 \text{ (bit-wide double)} = 128\text{-way parallelism}$
- Using only 1 serial path out of 128 parallel paths means that we are using $1/128 = 0.008$ or 0.8% of the available computational capabilities.

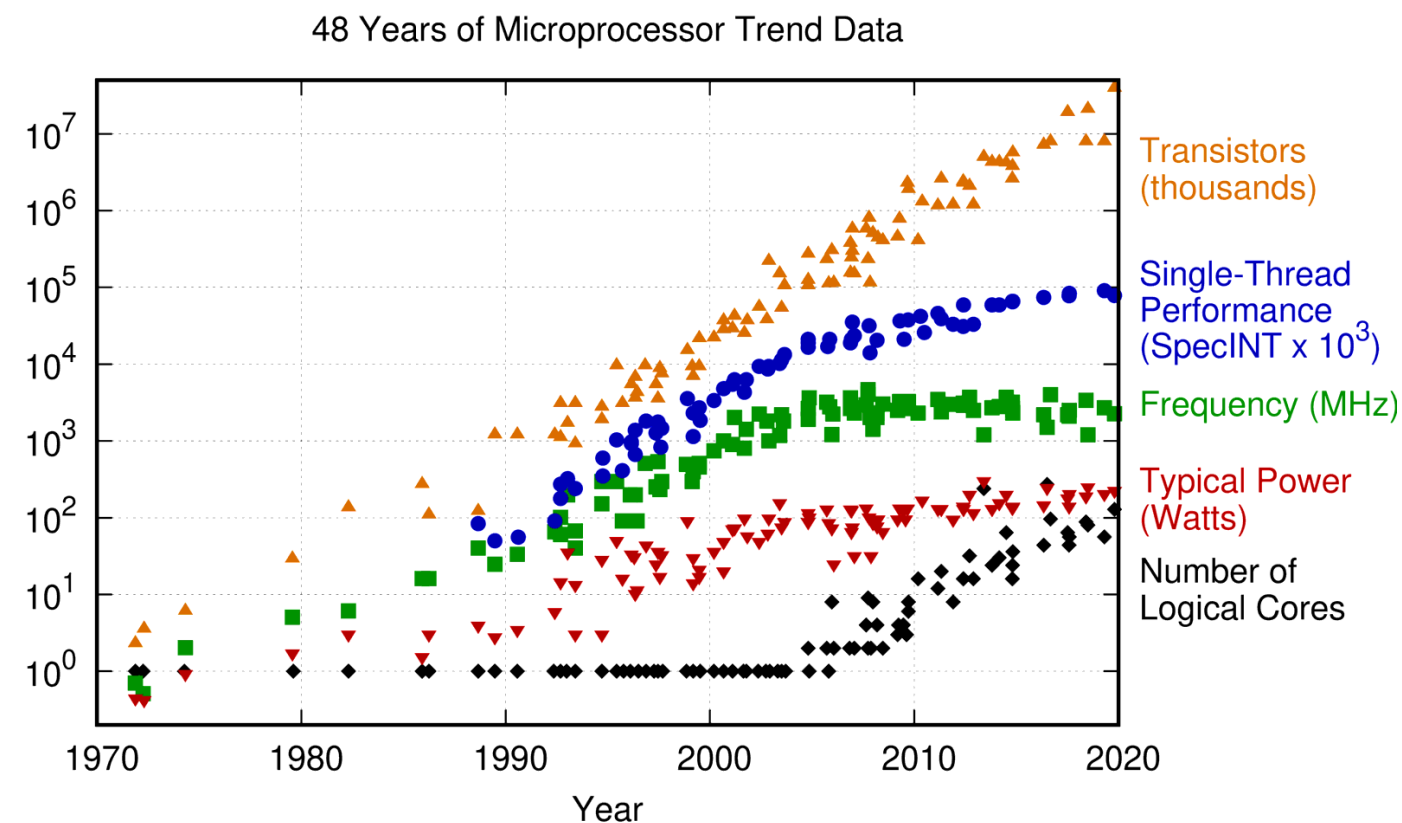
Note that this calculation doesn't take into account many real-world constraints...
it's hard to really exploit that 128-way parallelism

CPUs...

- The clock speed of a processor cannot be increased without overheating

But...

- More and more processors can fit in the same space
- Multicores are everywhere



...and all the rest

- Grid computing: collection of computer resources from multiple locations to reach a common goal.
- Cluster computing: loosely or tightly connected computers that work together so that, in many respects, they can be viewed as a single system.
- Cloud computing: a model for enabling ubiquitous network access to a shared pool of configurable computing resources.



...and all the rest

- Grid computing: collection of computer resources from multiple locations to reach a common goal.

Grids are a form of distributed computing whereby a “super virtual computer” is composed of many networked loosely coupled computers acting together to perform large tasks.

Grid computing is a collection of geographically distributed computers that work together so that, in many respects, they can be viewed as a single system.

- Cloud computing: a model for enabling ubiquitous network access to a shared pool of configurable computing resources.

...and all the rest

- Grid computing: collection of computer resources from multiple locations to reach a common goal.

Grids are a form of distributed computing whereby a “super virtual computer” is composed of many networked loosely coupled computers acting together to perform large tasks.

Grid computing is a collection of geographically distributed computers that work together so that, in many respects, they can be viewed as a single system.

The components of a cluster are usually connected to each other through LAN, with each node (computer used as a server) running its own instance of an operating system.

- Cloud computing: a model for enabling ubiquitous network access to a shared pool of configurable computing resources.

...and all the rest

- **Grid computing: collection of computer resources from multiple locations to reach a common goal.**

Grids are a form of distributed computing whereby a “super virtual computer” is composed of many networked loosely coupled computers acting together to perform large tasks.

Grid computing is a collection of geographically distributed computers that work together so that, in many respects, they can be viewed as a single system.

The components of a cluster are usually connected to each other through LAN, with each node (computer used as a server) running its own instance of an operating system.

- **Cloud computing: a model for enabling ubiquitous network access to a shared pool of configurable**

Cloud computing and storage solutions provide users and enterprises with various capabilities to store and process their data in third-party data centers.[2] It relies on sharing of resources to achieve coherence and economies of scale, similar to a utility (like the electricity grid) over a network.

Parallel vs. Distributed

- **Parallel computing:** provide performance.
 - In terms of processing power or memory;
 - To solve a single problem;
 - Typically: frequent, reliable interaction, fine grained, low overhead, short execution time.
- **Distributed computing:** provide convenience.
 - In terms of availability, reliability and accessibility from many different locations;
 - Typically: interactions infrequent, with heavier weight and assumed to be unreliable, coarse grained, much overhead and long uptime.

Parallel vs. Distributed

- **Parallel computing:** provide performance.
 - In terms of processing power or memory;
 - To solve a single problem;
 - Typically: frequent, reliable interaction, fine grained, low overhead, short execution time.
- **Distributed computing:** provide convenience.

We will not deal with distributed computing, in this course.

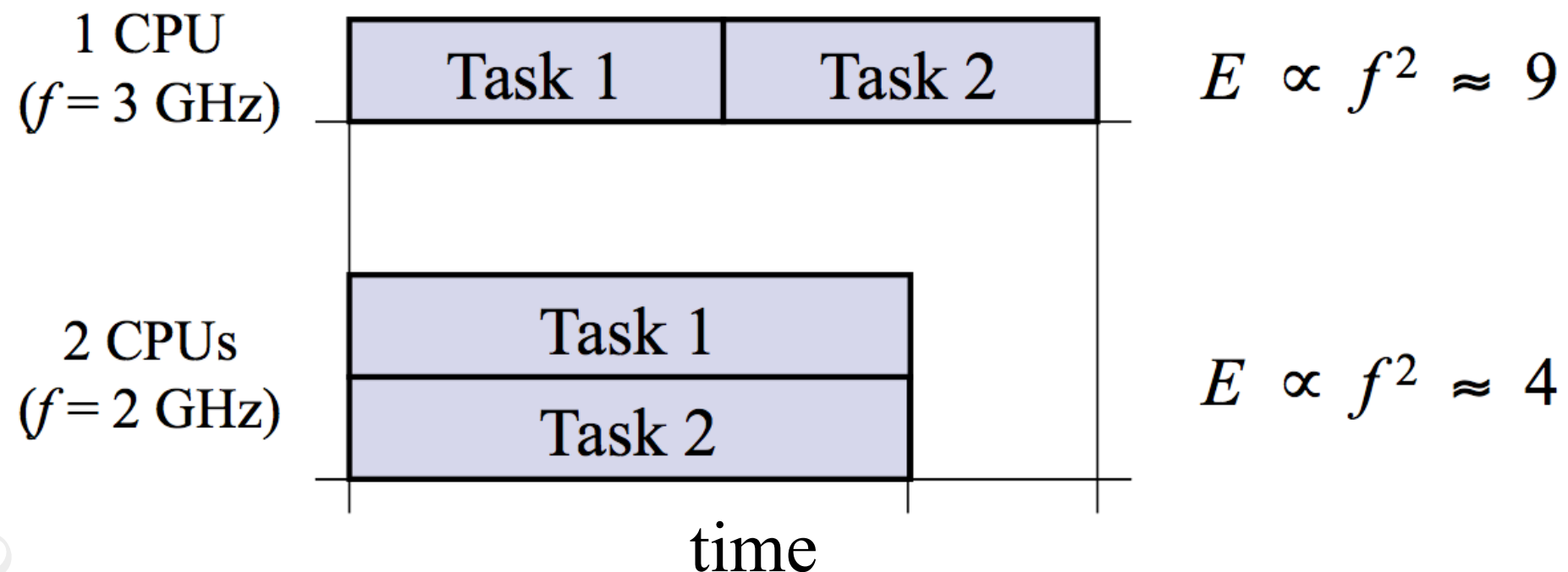
- Typically: interactions infrequent, with heavier weight and assumed to be unreliable, coarse grained, much overhead and long uptime.

Power consumption

- The power consumption of a CPU has become increasingly important.
- Energy is more suited to compare 2 CPUs (a faster one may consume more power but for much less time, resulting in less energy consumption...)
- In a CPU most of the energy is consumed switching transistor states. This so called *dynamic energy* is proportional to the square of the voltage V and frequency f , so power consumption is: $P \propto f V^2$
- The voltage required to operate the CPU at given frequency is, roughly proportional to the frequency ($P \propto f^3$).
- Increasing the frequency, the same amount of work can be finished earlier ($\sim$ inversely to the frequency), so the required energy is: $E \propto f^2$

Physical limitations

- As noted by Intel CTO in 2004: “[without revolutionary new technologies...] increased power requirements of newer chips will lead to CPUs that are hotter than the surface of the sun by 2010.”
- Parallelism can be used to conserve energy



Energy: CPUs vs GPUs

- GPUs need lot of power but thanks to increased parallelism they can reduce consumption.
- Example:
 - Intel's 16-core Xeon E5-4660 processor has a thermal design power of 120 W. Suppose we need 20 of these processors for 24 hours to run to completion. The estimated energy usage for your application is
 - $P = (20 \text{ Processors}) \times (120 \text{ W/Processors}) \times (24 \text{ hours}) = 57.60 \text{ kWh}$
 - Suppose we can port the application on 4 NVIDIA Tesla V100 GPUs running the same program in 24 hours. NVIDIA's Tesla V100 GPU has a maximum thermal design power of 300 W. The estimated energy usage for your application is
 - $P = (4 \text{ GPUs}) \times (300 \text{ W/GPUs}) \times (24 \text{ hrs}) = 28.80 \text{ kWh}$

Energy: CPUs vs GPUs

- GPUs need lot of power but thanks to increased parallelism they can reduce consumption.

There's also a cost reduction: maintaining a server with 4 GPUs is probably easier than keeping 5 servers with 4 CPUs

- Intel's 16-core Xeon E5-4660 processor has a thermal design power of 120 W. Suppose we need 20 of these processors for 24 hours to run to completion. The estimated energy usage for your application is
 - $P = (20 \text{ Processors}) \times (120 \text{ W/Processors}) \times (24 \text{ hours}) = 57.60 \text{ kWh}$
- Suppose we can port the application on 4 NVIDIA Tesla V100 GPUs running the same program in 24 hours. NVIDIA's Tesla V100 GPU has a maximum thermal design power of 300 W. The estimated energy usage for your application is
 - $P = (4 \text{ GPUs}) \times (300 \text{ W/GPUs}) \times (24 \text{ hrs}) = 28.80 \text{ kWh}$

Physical limitations

- Transistors are getting smaller (somehow the Moore's law is still valid), but they are not getting faster.
 - Reducing the gate size in planar MOSFET transistors leads to "leakage". There's need of new geometries (e.g. FinFET, Nanosheet).
- Signalling rates on chip are limited by the resistance and capacitance of the wires connecting the transistors. The smaller they get the thinner the wires (thus higher resistance) and the higher the capacitance.
 - The time delay for a wire depends on what is called the "RC time constant", which is the resistance multiplied by the capacitance.

Physical limitations

- Transistors are getting smaller (somehow the Moore's law is still valid), but they are not getting faster.
 - Reducing the gate size in planar MOSFET transistors leads to "leakage". There's need of new geometries (e.g. FinFET, Nanosheet).
- Signalling rates are limited by the parasitic capacitance of the wires connecting the transistors. The smaller they get the thinner the wires (thus higher resistance) and the higher the capacitance.

It is much more convenient to add cores, rather than increasing speed

 - The time delay for a wire depends on what is called the "RC time constant", which is the resistance multiplied by the capacitance.

Physical limitations

- Suppose we have to calculate in one second

```
for (i = 0; i < ONE_TRILLION; i++)  
    z[i] = x[i] + y[i];
```

- Then we have to perform 3×10^{12} memory moves per second
- If data travels at the speed of light (3×10^8 m/s) between the CPU and memory, and r is the average distance between the CPU and memory, then r must satisfy
 - $3 \times 10^{12} r = 3 \times 10^8 \text{ m/s} \times 1 \text{ s}$ which gives $r = 10^{-4}$ meters
- To fit the data into a square so that the average distance from the CPU in the middle is r , then the length of each (square) memory cell will be
 - $2 \times 10^{-4} \text{ m} / (\sqrt{3 \times 10^6}) = 10^{-10} \text{ m}$ which is the size of a relatively small atom!

Instead, the computation can be parallelized to complete it within the required time limit

Important factors

- Important considerations in parallel computing
 - Physical limitations: the speed of light, CPU heat dissipation
 - Scalability: allows data to be subdivided over CPUs to obtain a better match between algorithms and resources to increase performance
 - Memory: allow aggregate memory bandwidth to be increased together with processing power at a reasonable cost
 - Economic factors: cheaper components can be used to achieve comparable levels of aggregate performance

Sometimes parallel is bad

- Bad parallel programs can be worse than their sequential counterparts:
 - Slower: because of communication overhead
 - Scalability: some parallel algorithms are only faster when the problem size is very large
- Understand the problem and use common sense!
- Moreover: not all problems are amenable to parallelism



Types of parallelism

Bit-Level Parallelism

- It is a form of parallel computing based on increasing processor word size.
- If an 8-bit computer wants to add two 32-bit numbers, it has to do it as a sequence of 8-bit operations. By contrast, a 32-bit computer can do it in one step, handling each of the 4 bytes within the 32-bit numbers in parallel.
- Though, it's unlikely that we'll have 128-bit computers soon.

Instruction-Level Parallelism

- Modern CPUs are highly parallel, using techniques like *pipelining*, *out-of-order execution*, and *speculative execution*.

- Pipelining*: execution of multiple instructions can be partially overlapped.

Instr. No.	Pipeline Stage						
	IF	ID	EX	MEM	WB		
1							
2							
3							
4							
5							
Clock Cycle	1	2	3	4	5	6	7

- O-o-O execution*: instructions execute in any order that does not violate data dependencies. Its benefit grows as the pipeline gets deeper (and speed difference between CPU and RAM/Cache increases).
- Speculative execution*: used to reduce cost of conditional branch instructions in pipelined CPUs. Instructions are scheduled ahead of the determination of the need of their execution. It may use *branch prediction*.

Instruction-Level Parallelism

- Modern CPUs are highly parallel, using techniques like *pipelining*, *out-of-order execution*, and *speculative execution*.

- On modern (multicore) CPUs instructions will not be executed in the same order in which you write them

Instr. No.	Pipeline Stage						
	IF	ID	EX	MEM	WB		
1							
2							
3							
4							
5							
Clock Cycle	1	2	3	4	5	6	7

- O-o-O execution*: instructions execute in any order that does not violate data dependencies. Its benefit grows as the pipeline gets deeper (and speed difference between CPU and RAM/Cache increases).
- Speculative execution*: used to reduce cost of conditional branch instructions in pipelined CPUs. Instructions are scheduled ahead of the determination of the need of their execution. It may use *branch prediction*.

Instruction-Level Parallelism

- Modern CPUs are highly parallel, using techniques like *pipelining*, *out-of-order execution*, and *speculative execution*.

- On modern (multicore) CPUs instructions will not be executed in the same order in which you write them

Instr. No.	Pipeline Stage						
1	IF	ID	EX	MEM	WB		
2		IF	ID	EX	MEM	WB	
3			IF	ID	EX	MEM	WB
4				IF	ID	EX	MEM
5					IF	ID	EX
Clock Cycle	1	2	3	4	5	6	7

- O-o-O execution*: instructions execute in any order that does not violate data dependencies. Its benefit grows as

DIVD f6, f2, f4 ; slow... latency
 ADDD f0, f6, f8 ; must wait for f6
 SUBD f12, f8, f14 ; could be executed before DIVD or ADDD

- Speculative execution*: used to reduce cost of conditional branch instructions in pipelined CPUs. Instructions are scheduled ahead of the determination of the need of their execution. It may use *branch prediction*.

Data Parallelism

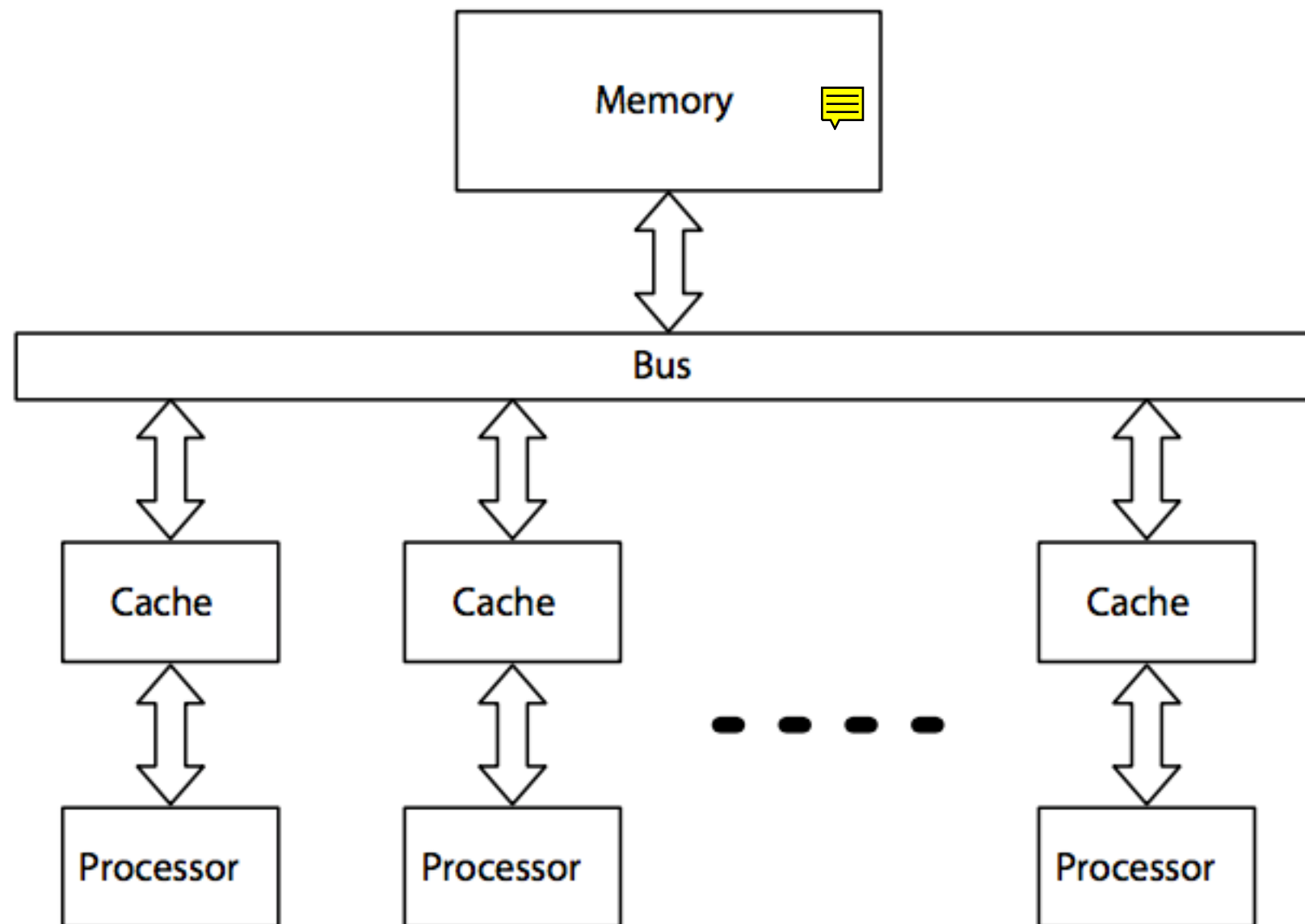
- Also known as SIMD (Single Instruction, Multiple Data)
- This architecture performs same operations on a large quantity of data in parallel.
- Multimedia processing is a good candidate for this type of architecture (i.e. GPUs).



Task-Level Parallelism

- There are two important variations, related to the underlying memory model: shared vs. distributed.
- **Shared memory** multiprocessors: each CPU accesses any memory location, IPC is done through memory.
- **Distributed memory** system: each CPU has its own local memory, IPC is done through a network.
- Shared memory systems are simpler but do not scale after a certain number of processors: Distributed systems are the way to go for fault-tolerant systems.

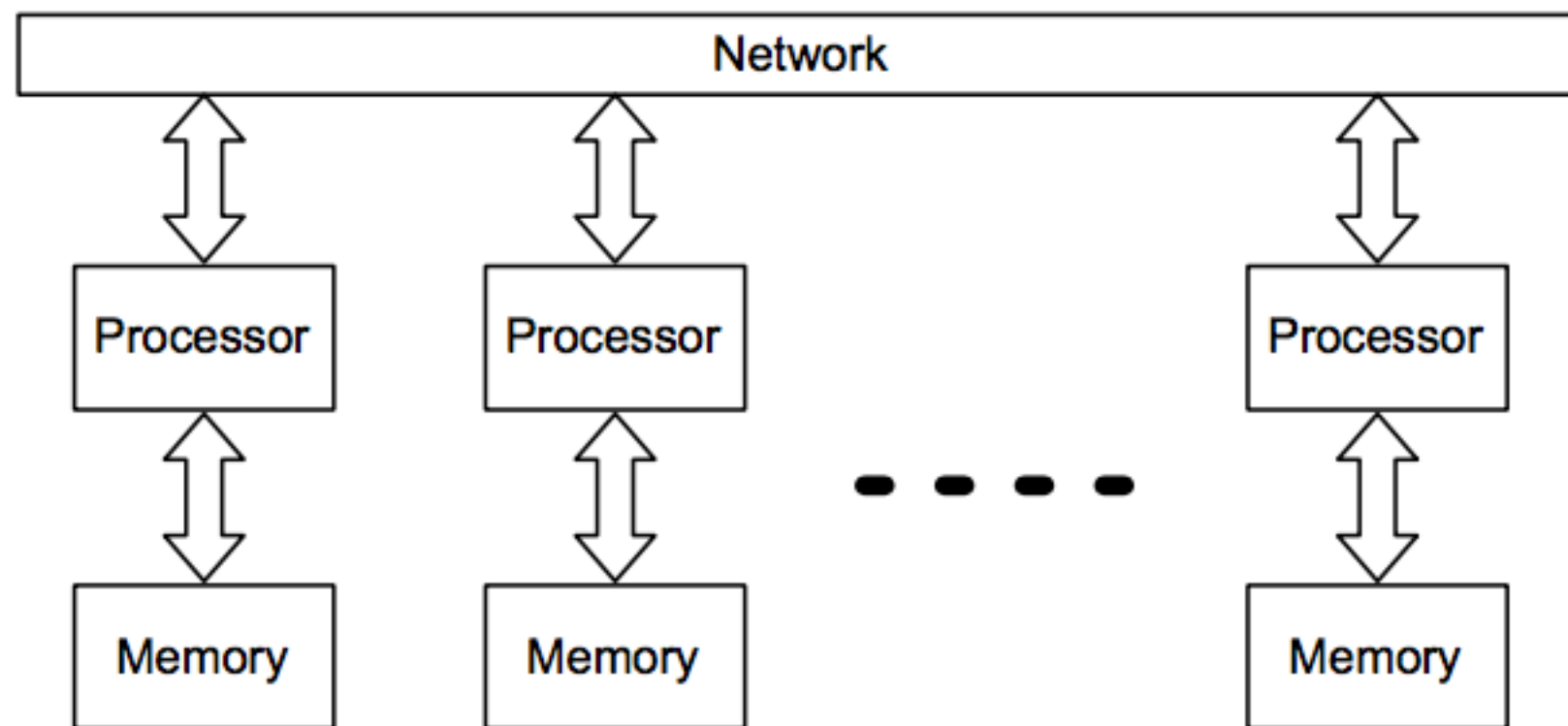
Shared memory system



- Natural extension of sequential computer: all memory can be referenced (single address space). Hardware ensures memory coherence.
- Good:
 - Easier to use
Through multi-threading
- Bad:
 - Easier to create faulty programs
Race conditions
 - More difficult to debug
Intertwining of threads is implicit



Distributed memory system

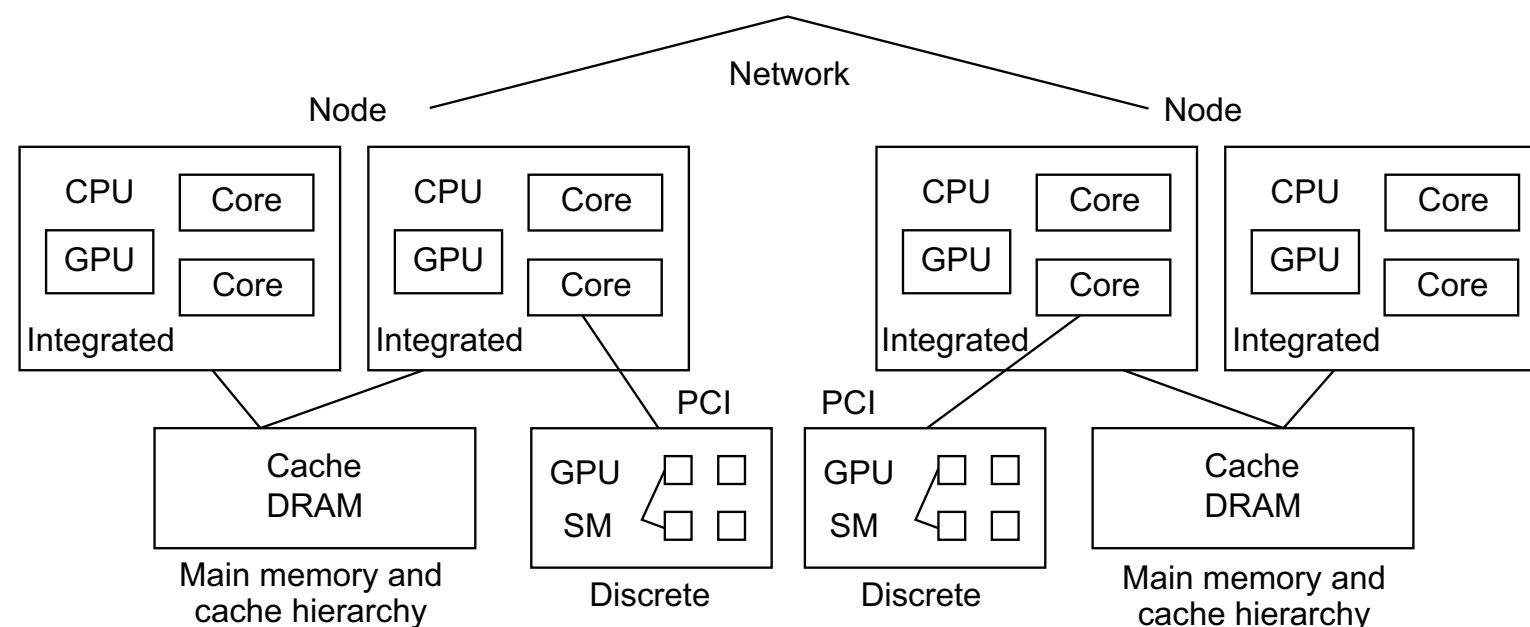


- Processors can only access their own memory and communicate through messages.
- Requires the least hardware support.
- Easier to debug. 
 - Interactions happens in well-defined program parts
 - The process is in control of its memory!
- Cumbersome communication protocol is needed
 - Remote data cannot be accessed directly, only via request.



Heterogeneous systems

- Today we commonly use heterogeneous parallel systems that combine:
 - Shared memory system
 - Distributed memory system 
 - Accelerator devices (i.e. distributed systems within a single unit)



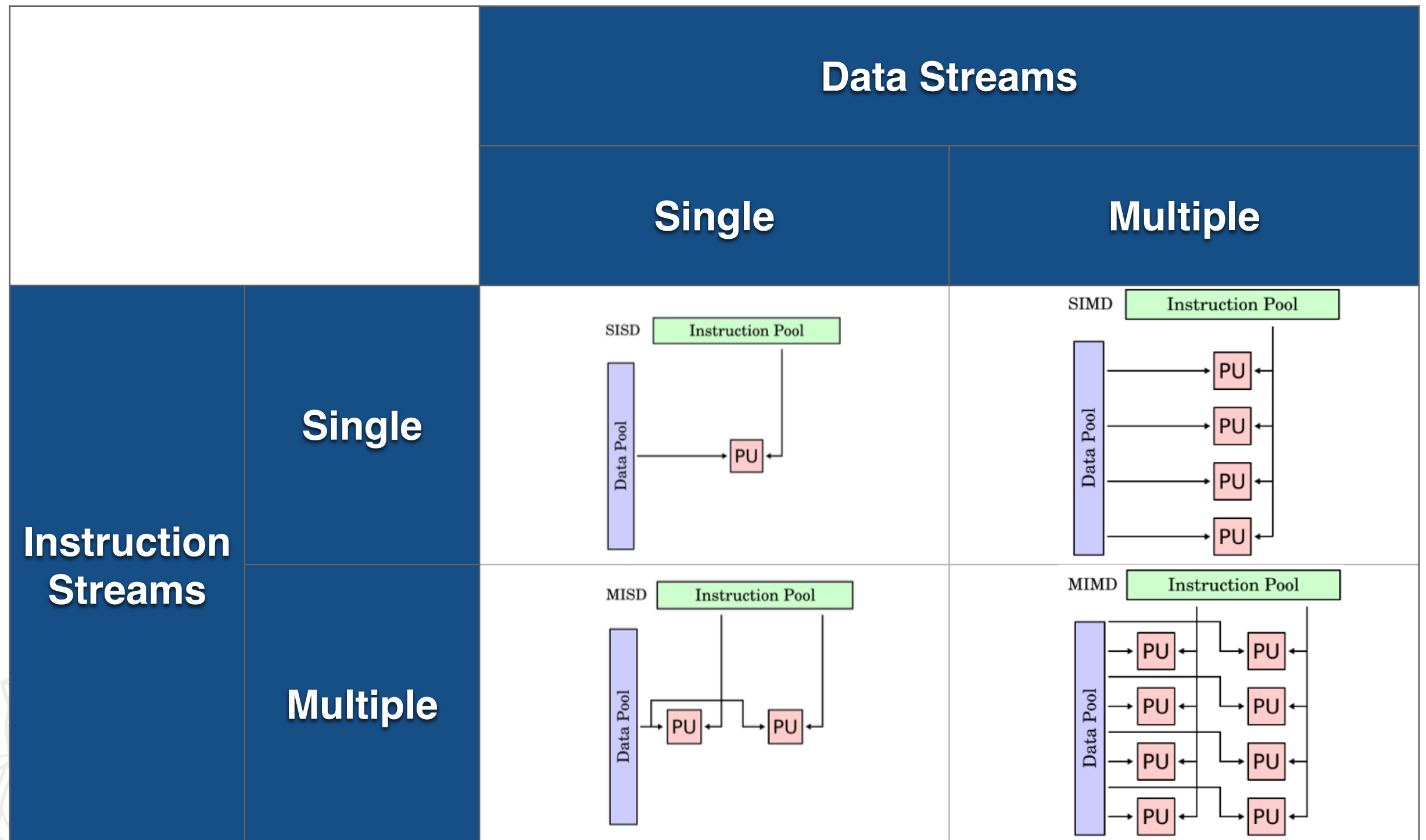
Flynn's taxonomy

		Data Streams	
		Single	Multiple
Instruction Streams	Single	SISD e.g. Intel Pentium 4	SIMD SSE instructions of x86
	Multiple	MISD	MIMD e.g. Intel Xeon Phi

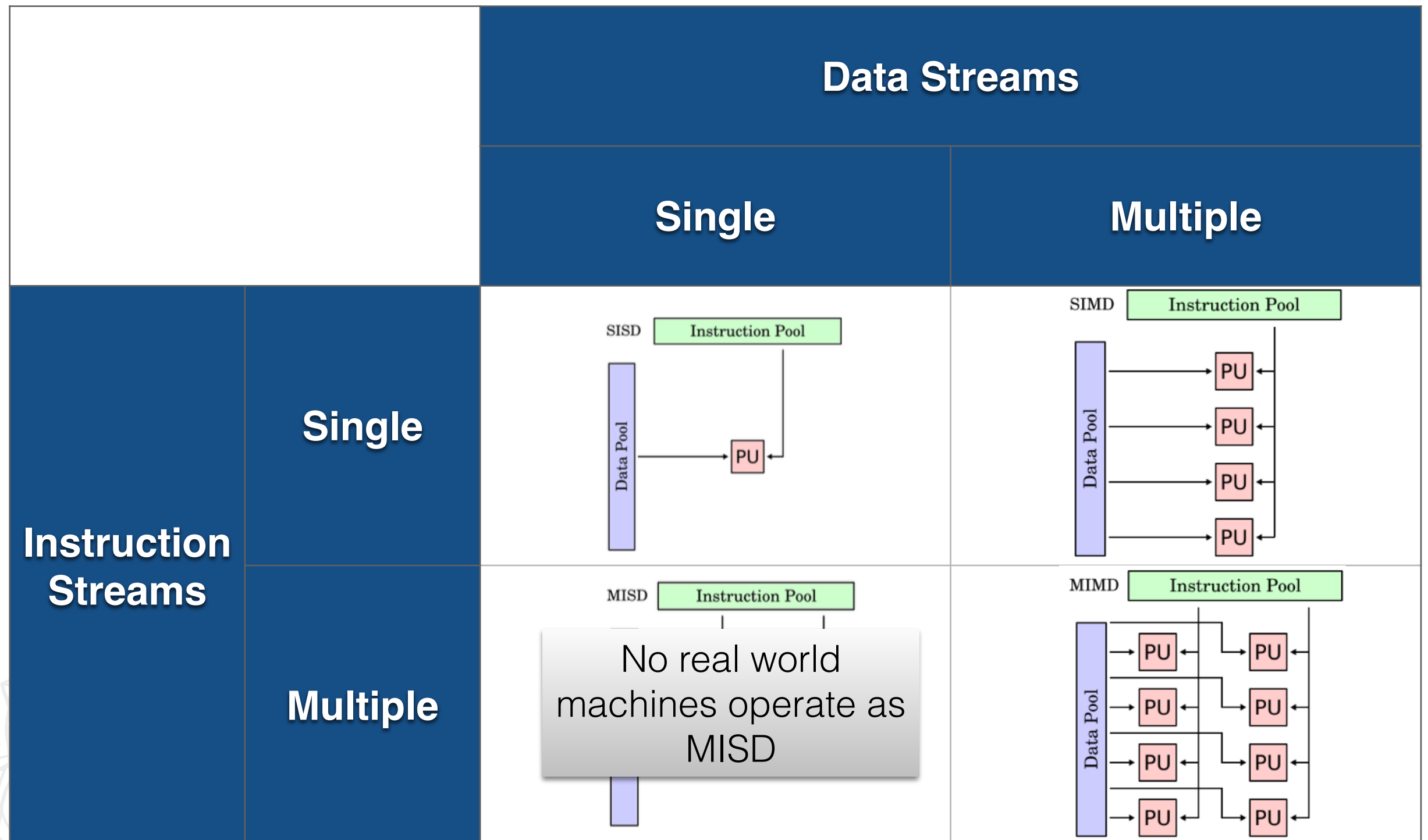
Flynn's taxonomy

- The taxonomy has been expanded with **SPMD** (single program, multiple data).
- Tasks are split up and run simultaneously on multiple processors with different input in order to obtain results faster.
- SPMD could be considered a parallel program on a MIMD computer.

Flynn's taxonomy



Flynn's taxonomy



Systolic array

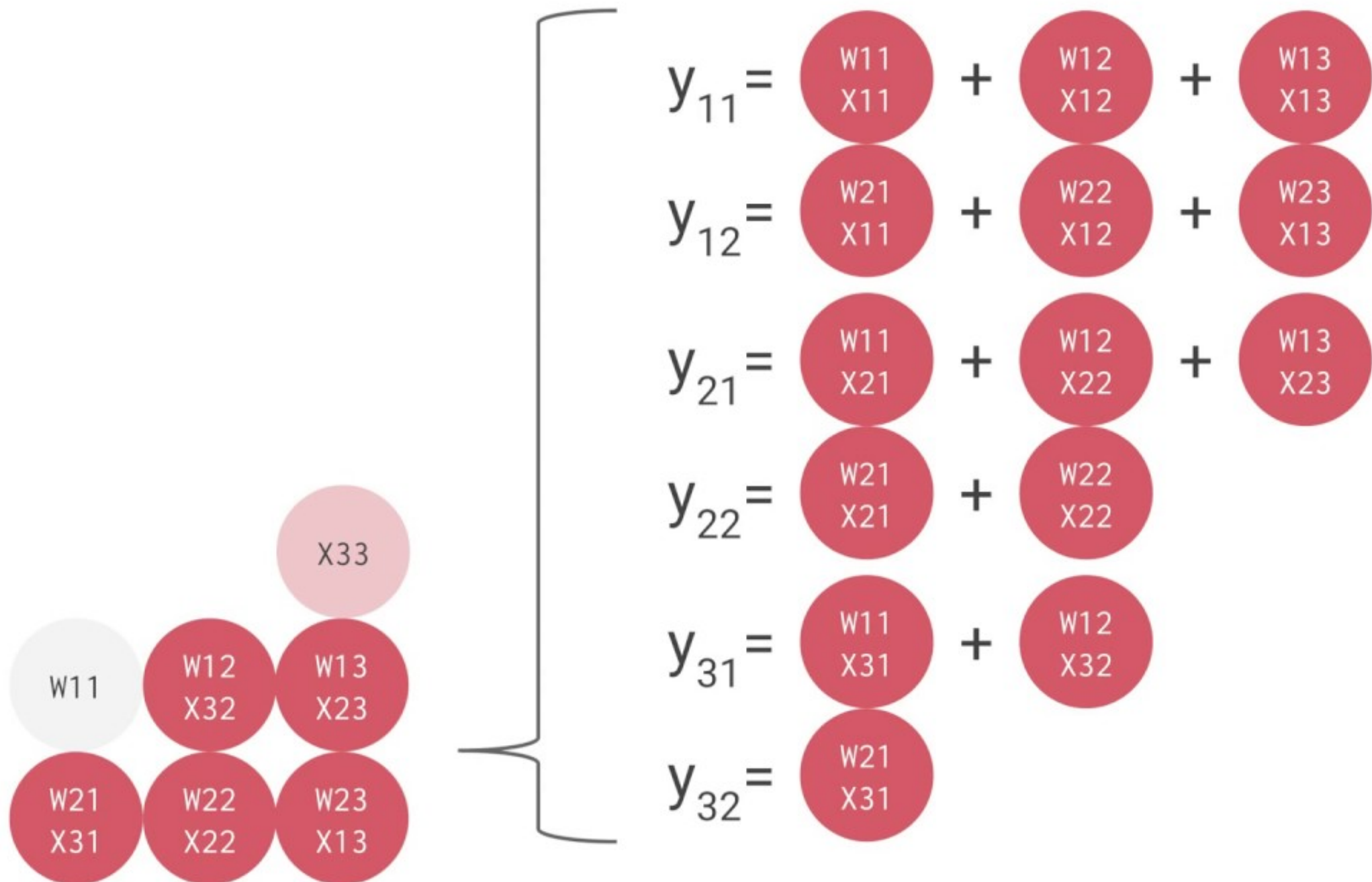
- A systolic array typically consists of a large monolithic network of primitive computing nodes (data processing units - DPUs) which can be hardwired or software configured for a specific application.
- Parallel input data enters this architecture flowing through the nodes that process it as it processes through the node as *waves*. The name of the architecture was picked as it resembles the systolic movement of the heart.
- Sometimes it is referred as an example of MISD but this is questionable as the nodes operate on different data.

Systolic array

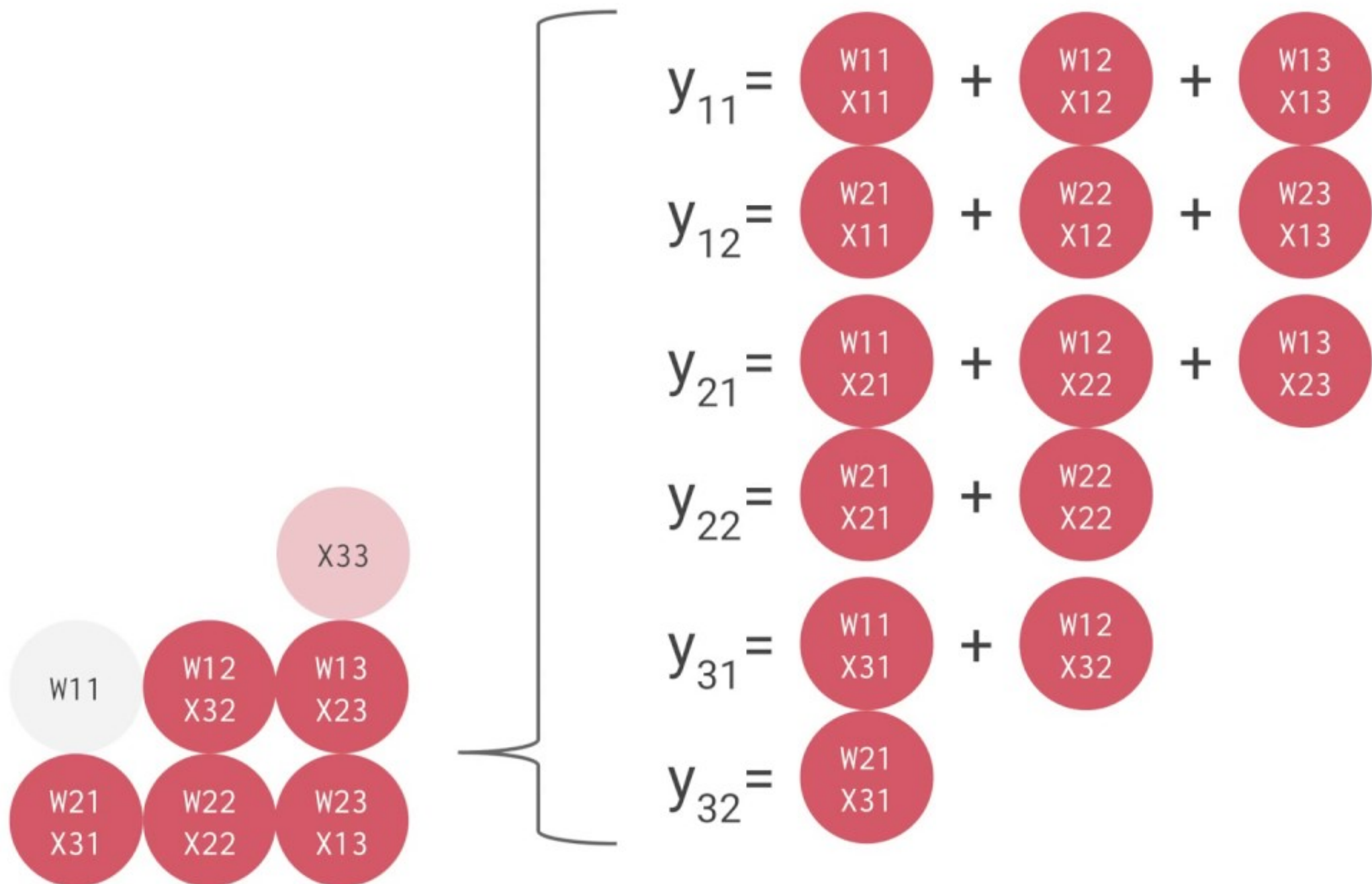
- Now commonly used to accelerate deep learning and computer vision tasks, implementing matrix-matrix multiplications
 - Google TPUs use systolic arrays
 - NVIDIA tensor cores (sort of)



Systolic array



Systolic array





UNIVERSITÀ
DEGLI STUDI
FIRENZE

Parallel architectures

RAM model

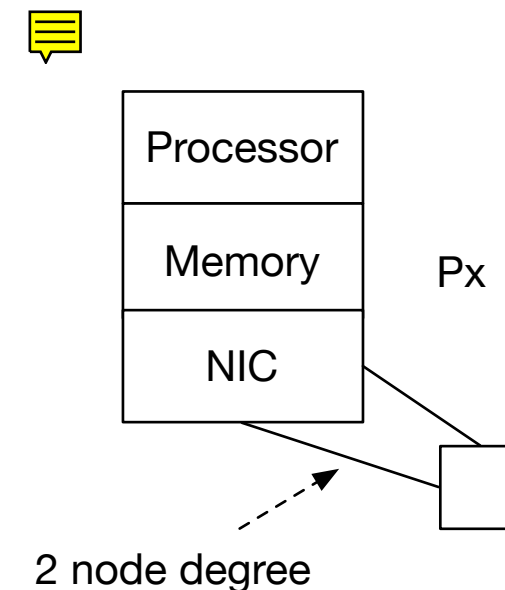
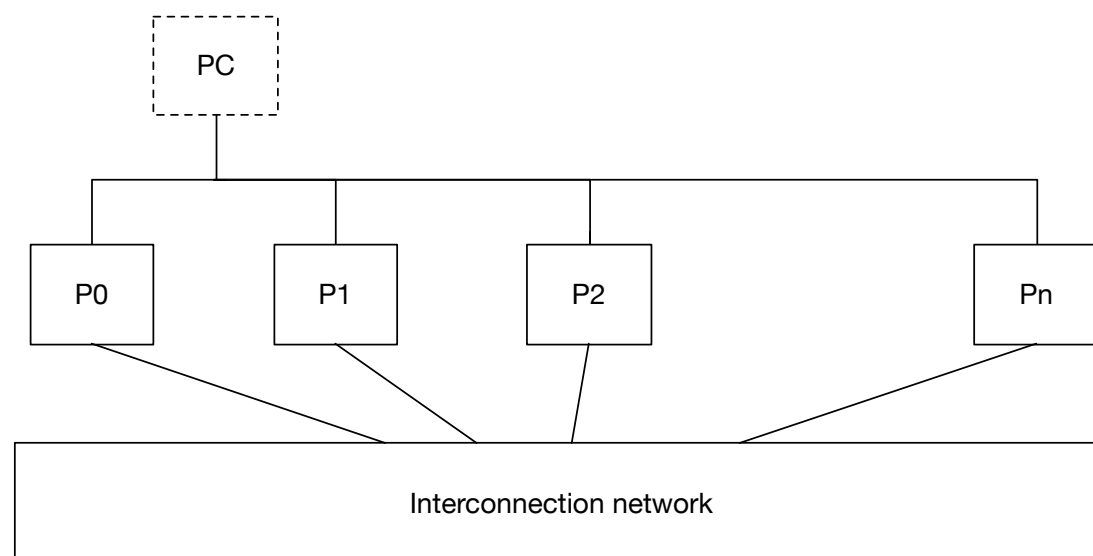
- Random Access Machine: is an abstract model for a sequential computer
- It models a device with an instruction execution unit and unbounded memory.
 - Memory stores program instructions and data.
 - Any memory location can be referenced in 'unit' time
 - The instruction unit fetches and executes an instruction every cycle and proceeds to the next instruction.
- Today's computers depart from RAM, but function as if they match this model.

PRAM model

- Parallel Random Access Machine: abstract model for parallel computer
- It models a device with an unspecified number of instruction execution units and global memory of unbounded size that is uniformly accessible to all processors
 - It fails by misrepresenting memory behavior.
 - Impossible to realize the unit-time single memory image when multiple exec. units access the same location
- Bad memory modeling leads to wrong evaluation of algorithms: PRAM's performance predictions are not observed in real computers !

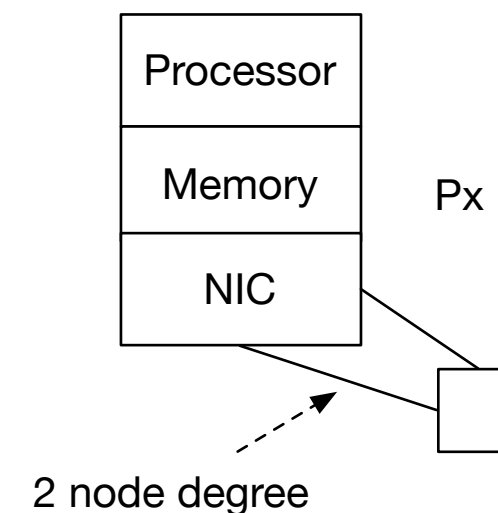
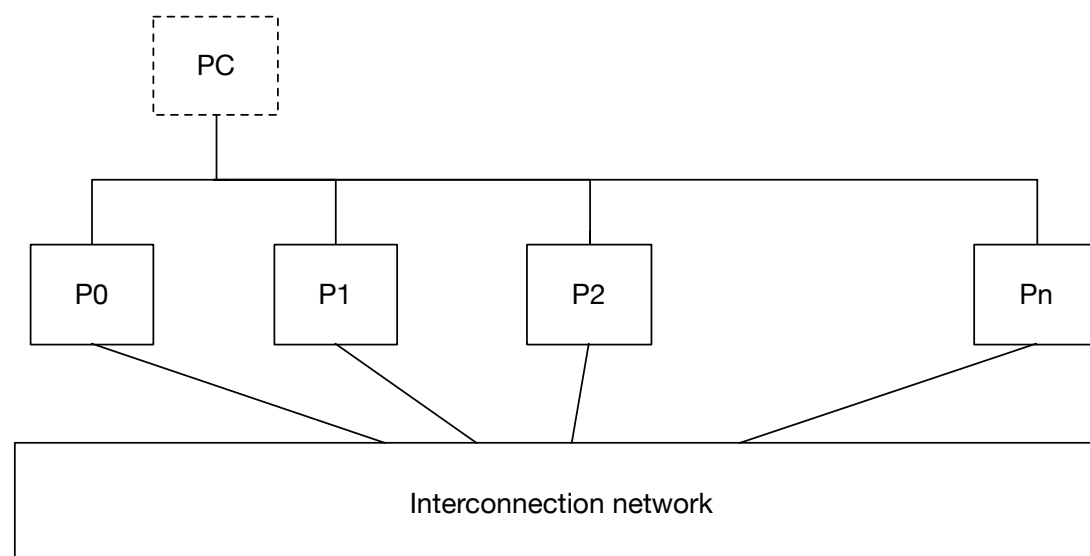
CTA model

- Candidate Type Architecture: abstract model for parallel computer
- Explicitly separates two types of memory references: inexpensive local references and expensive non-local references



Both shared and distributed memory architectures described before are correctly modeled in this way

- Candidate Type Architecture: abstract model for parallel computer
- Explicitly separates two types of memory references: inexpensive local references and expensive non-local references



Memory Latency λ

- Memory Latency = delay required to make a memory reference. Non local memory latency is indicated with λ .
- Relative to processor's local memory latency, $\approx$ unit time $\approx$ one word per instruction
 - Variable, due to cache mechanisms etc.
- λ has values 2-5 orders of magnitude larger than local memory reference 
- Sometimes it is better to recalculate globals locally (e.g. random number) 